Object Oriented Programming (CSEG1044)
Unit 01 – Lecture 01 Notes
OOP Paradigm + Principles + Modelling

Tofik Ali

February 26, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 01) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **2**
5.1 Why OOP? (vs procedural) . . . 2
5.2 The 4 OOP principles (with intuition) . . . 3
5.2.1 Abstraction . . . 3
5.2.2 Encapsulation . . . 3
5.2.3 Inheritance . . . 3
5.2.4 Polymorphism . . . 4
5.3 How to model a real-world entity . . . 4

6 Worked Example: `BankAccount` (Encapsulation + Invariants) **4**

7 In-Class Exercises (with brief solutions) **5**

8 Self-Study Practice (no solutions) **5**

9 Checkpoint Questions (with sample answers) **6**

1 Lecture Overview

This lecture starts the course with two questions:

1. **Why do we use OOP?** (what problems it solves compared to purely procedural code)
2. **How do we think in OOP?** (principles + modelling a real-world problem into classes and objects)

We will connect ideas to simple, beginner-friendly Java examples (no advanced syntax yet).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed February 26, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed February 26, 2026).

2 Syllabus Alignment (Unit 01)

Unit 01: Introduction to OOPs

- OOP history and evolution
- OOP principles
- Modelling a real-world entity

3 Learning Outcomes

After this lecture, you should be able to:

- Explain (in your own words) why OOP became popular for large software systems.
- Define and differentiate **abstraction**, **encapsulation**, **inheritance**, and **polymorphism**.
- Convert a short problem statement into a small class design (classes, fields, methods, responsibilities).

4 Key Terms (Quick)

- **Class**: A blueprint (type) describing data (fields) and operations (methods).
- **Object**: A runtime instance of a class (has identity + state + behavior).
- **State**: The data inside an object (values of its fields at a time).
- **Behavior**: What an object can do (methods).
- **Invariant**: A rule that must always remain true for an object (e.g., `balance >= 0`).
- **Interface (idea)**: A public view/contract of what operations are allowed (we will formalize this later in Java).

5 Detailed Notes

5.1 Why OOP? (vs procedural)

Procedural programming organizes software around *procedures/functions* that operate on data. This is great for small programs, but as the program grows, a few problems become common:

- **Shared state becomes risky:** if many functions can modify the same data, it becomes hard to guarantee correctness.

- **Tight coupling:** one change in data structure often forces changes in many functions.
- **Poor "real-world" mapping:** business concepts (student, course, account, booking) are not first-class; you must manually manage relationships.

Object-Oriented Programming (OOP) organizes software around *objects*. An object keeps its data together with the operations that are allowed on that data. That makes it easier to:

- **Divide responsibilities** (each class owns a small part of the system).
- **Protect invariants** (only methods can change internal data).
- **Reuse designs** (inheritance, composition, and interfaces).
- **Build large systems** by combining small, understandable pieces.

Rule of thumb. In OOP, *data is not "free" to modify.* Most fields should be private and updated through methods that validate.

5.2 The 4 OOP principles (with intuition)

5.2.1 Abstraction

Abstraction means focusing on *what* something does, not *how* it does it. We hide irrelevant details so that the user of a class sees a simple model.

Example. You know a `BankAccount` supports `deposit()` and `withdraw()`. You do not care about internal storage details.

Pitfall. If you expose internal details (public fields), users can bypass rules and break the model.

5.2.2 Encapsulation

Encapsulation means bundling data with methods and controlling access to protect invariants. In Java this usually means `private` fields + public methods.

Example. If `balance` is private, only `deposit()` / `withdraw()` can change it, so we can ensure `balance >= 0`.

Pitfall. "Getters/setters for everything" is not real encapsulation if setters allow invalid states (e.g., `setBalance(-100)`).

5.2.3 Inheritance

Inheritance models an **is-a** relationship: a child class is a specialized form of a parent class. It enables reuse of common code *when the relationship is truly is-a.*

Example. `SavingAccount` is-a `BankAccount` (it has all account behaviors and adds interest calculation).

Pitfall. Do not use inheritance only to reuse code. If it is not is-a, prefer composition (has-a).

5.2.4 Polymorphism

Polymorphism means "many forms". In Java, the most important form is **runtime polymorphism**: when a parent reference points to a child object, the overridden method that runs is chosen at runtime.

Example (idea). A variable of type `BankAccount` can hold a `SavingAccount`; calling `withdraw()` executes the child's override if it exists.

Pitfall. Confusing **overloading** (same method name, different parameters) with **overriding** (same signature in subclass).

5.3 How to model a real-world entity

When you get a problem statement, you are doing **modelling**: converting real-world concepts into a class design. One practical step-by-step method:

1. **Read scenarios**: list 3–5 user actions ("deposit money", "book a room").
2. **Extract nouns**: candidate classes/entities (Account, Customer, Booking).
3. **Extract verbs**: candidate methods/operations (deposit, book, cancel).
4. **Define responsibilities**: what each class owns/does (avoid "God" classes).
5. **Write invariants**: rules that must never break (balance non-negative, booking has start $<$ end).
6. **Validate with examples**: run your design on sample inputs mentally.

CRC cards (quick idea). For each class, write: **C**lass name, **R**esponsibilities, and **C**ollaborators. This simple technique prevents one class from doing everything.

6 Worked Example: `BankAccount` (Encapsulation + Invariants)

Goal

Model a bank account so that the invariant `balance >= 0` is always true.

Code

```
public class BankAccount {
    private final String accountNo;
    private double balance; // invariant: balance >= 0

    public BankAccount(String accountNo, double openingBalance) {
        if (accountNo == null || accountNo.isBlank())
            throw new IllegalArgumentException("accountNo required");
        if (openingBalance < 0)
            throw new IllegalArgumentException("openingBalance cannot be negative");

        this.accountNo = accountNo;
        this.balance = openingBalance;
```

```java
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
        if (amount > balance) throw new IllegalArgumentException("insufficient balance");
        balance -= amount;
    }
}
```

What to notice

- Encapsulation: `balance` is `private`.
- Invariant protection: constructor and methods validate inputs.
- Abstraction: users do not directly "set balance"; they request operations.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed February 26, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed February 26, 2026).

7 In-Class Exercises (with brief solutions)

1. **Model a `Student` entity.** Identify 3 fields and 3 methods.

   Sample: fields = `rollNo, name, cgpa`; methods = `updateCgpa(), enrollCourse(), printTranscript()`.

2. **Find the invariant.** For each, write one invariant:

   (a) Rectangle **(b)** Inventory item **(c)** Booking.

   Sample: Rectangle: width/height > 0; Booking: startDate $<$ endDate.

3. **Is-a or has-a?** Decide if it should be inheritance or composition:

   (a) Car and Engine **(b)** Dog and Animal.

   Answer: Car has-a Engine (composition). Dog is-a Animal (inheritance).

8 Self-Study Practice (no solutions)

1. Convert a real-world object (Library Book, ATM Card, Classroom) into a class with fields + methods + 1 invariant.

2. Write a 5–7 line paragraph: "Why OOP helps in large software projects" using one example.
3. Modify `BankAccount` to add `transferTo(BankAccount other, double amount)`.

9 Checkpoint Questions (with sample answers)

Q1. Why do we keep fields private in OOP?

Sample answer: Private fields prevent external code from putting an object into an invalid state. Changes happen only through methods, so we can validate inputs and preserve invariants.

Q2. What is the difference between abstraction and encapsulation?

Sample answer: Abstraction is about simplifying the model (what is exposed); encapsulation is about protecting the internal state and controlling access (how it is protected).

Q3. Give one example of "is-a" and one example of "has-a".

Sample answer: Dog is-a Animal (inheritance). Car has-a Engine (composition).